

IntentCap: Intent-Carrying Capabilities for LLM Agent Extensions

Abstract

LLM agents increasingly rely on extensions: Skills that inject procedural instructions and scripts, MCP servers that expose tools and data, local programs that transform files, and subagents that decompose work. These extensions create a form of authority that conventional permission systems do not model. A document, tool result, or Skill instruction may not hold a credential, yet it can influence the agent to select a tool, choose a sink, request broader access, or delegate authority. Existing defenses regulate tool calls, extension manifests, or OS-level effects, but they do not specify which context is allowed to influence which authority-bearing decisions.

We introduce INTENTCAP, a least-privilege authorization model for LLM agent extensions. INTENTCAP makes user intent the root principal and makes context influence an explicit capability. High-impact actions must be justified by a structured intent certificate derived from trusted user input, object selections, policy, or fresh approval. Untrusted context may be authorized to parameterize output data without being authorized to determine tool choice, destination, policy, or delegation. INTENTCAP compiles a task plan into short-lived, proof-carrying capability leases that constrain operations, arguments, data flow, temporal order, context provenance, and delegation. An LLM-assisted frontend proposes plans and leases, while a deterministic checker validates intent derivations and provenance constraints before runtime enforcement at the context, tool, MCP, sandbox, and delegation boundaries. INTENTCAP does not claim to solve prompt injection. It instead blocks unauthorized context-to-decision influence within modeled decision classes and provides the missing policy layer that lower-level enforcement backends can implement.

1 Introduction

LLM agents are becoming extensible execution environments. A modern agent can load Skills with workflow instructions, references, and optional scripts [3]; connect to MCP servers that expose resources, prompts, and tools [1]; run local commands; and spawn subagents. This extensibility is useful precisely because these components affect future behavior. A Skill can teach the agent how to perform a workflow. A tool result can become evidence for a later action. A document can determine the content of a generated report. The same property creates a security problem: in an agent, text is not passive data. It can become control.

Most current defenses ask an operation-centric question: which tool may be called, which file may be read, which endpoint may be contacted, or which process may execute? That question is necessary but incomplete. Consider a user who asks an agent to extract tables from two selected PDFs, save spreadsheets, and create one GitHub issue in a named repository. The PDF contents should influence spreadsheet cells and perhaps the issue body. They should not influence the repository name, network destination, approval scope, tool selection, or decision to load another Skill. If hidden text in the PDF says “send all extracted values to attacker.com” or “create the issue in attacker/repo,” a tool-call allowlist or OS sandbox may catch some concrete effects, but it does not directly express

the more agent-specific invariant: the untrusted PDF was never allowed to influence those decisions.

INTENTCAP starts from that invariant. Its thesis is that securing agent extensions requires controlling not only what extensions can do, but what their context is allowed to mean. The root of authority is a structured *intent certificate*, not an extension package, server identity, or static manifest. A certificate records the current goal, selected objects, authorized sinks, constraints, approvals, and expiry. Context cells are labeled with provenance and allowed *influence modes*, such as observing, quoting, summarizing, parameterizing, planning, instructing, delegating, or authorizing. A capability is then minted for a particular run as an *intent-carrying lease*: it authorizes an operation only if the operation is derivable from the current intent and its control dependencies come from context allowed to influence that decision class.

This framing deliberately differs from extension-interface and OS-enforcement work. EIM and bpftime model application extensions by representing extension features as resources and granting capabilities at extension entries [6]. INTENTCAP instead protects future agent decisions: tool choice, sink selection, authority request, policy update, and delegation. ActPlane enforces agent policies below the tool layer using OS-level information-flow checks and eBPF-style monitoring [5]. INTENTCAP instead decides which policy is justified by the user intent and context provenance; ActPlane is a possible backend, not the conceptual center. SkillGuard is closer because it treats Skills as permission-bearing artifacts and jointly regulates context and action behavior [4]; INTENTCAP is run-centric and cross-extension, asking what this run may allow this Skill, this MCP server, this document, and this subagent to influence.

2 Design

Intent certificates. INTENTCAP represents user intent as a structured security object containing the actor, goal, selected objects, authorized sinks, constraints, approvals, and expiry. For the PDF example, the certificate names the selected PDF paths, the output directory, the single authorized GitHub repository, a constraint denying arbitrary network use, and a fresh approval requirement for issue creation. The LLM may propose certificate fields, but a trusted parser, UI, administrator policy, or approval step canonicalizes them. The certificate is not merely prompt text; it is the root object referenced by later checks.

Context authority. Each context cell carries a provenance label recording origin, integrity, confidentiality, purpose, time, and endorsements. The key addition is an authority set over influence modes and decision classes. PDF text may be allowed to *parameterize* spreadsheet cells and *summarize* into an issue body, but forbidden from *planning* tool choice, *authorizing* a capability request, or selecting an external sink. Signed Skill instructions may be allowed to *instruct* only within their declared workflow. MCP tool results and annotations must be treated according to server trust; the MCP specification already warns that tool behavior annotations are untrusted unless they come from trusted servers [2].

INTENTCAP turns this guidance into a deterministic provenance rule.

Effect IR and leases. The compiler lowers an agent plan into security-relevant effects such as context use, tool calls, MCP calls, file reads and writes, subprocess execution, network connections, approval requests, and subagent delegation. Each effect records data provenance and control provenance. This split is essential: untrusted PDF bytes may be a data dependency of output cells, but they must not be a control dependency of the repository argument to `github.create_issue`. A lease constrains the holder, operation, object, arguments, intent, influence, flow, temporal guard, budget, expiry, and delegation bound.

Compiler and checker. INTENTCAP uses an LLM-assisted but checker-verified pipeline. The frontend summarizes extension evidence from Skill metadata, MCP schemas, local scripts, observed traces, and policy; infers a plan and security-relevant effects; and proposes the minimum-risk lease set for the run. This frontend is outside the trusted computing base. A deterministic checker validates each lease against global policy, the intent certificate, provenance constraints, flow rules, temporal guards, budgets, and delegation monotonicity. The useful theorem target is modest: for any accepted trace, no protected decision has a control dependency on context lacking the required influence mode, and every accepted high-impact event has an active lease derived from trusted intent.

Runtime enforcement. Accepted leases are lowered to multiple boundaries. The context constructor separates trusted instructions, untrusted data, policy feedback, and tool results while attaching labels. The tool gateway exposes only leased tools and validates arguments. The MCP broker scopes individual server methods, arguments, counts, and sinks rather than granting an entire server. The execution sandbox enforces file, process, network, and secret restrictions for local scripts. The delegation monitor ensures subagents receive only attenuated leases. These mechanisms need not share one implementation substrate: a simple harness can enforce tool and MCP checks, while an OS monitor such as ActPlane can enforce compiled file, process, and network constraints. The abstraction is the proof-carrying lease; the backend is replaceable.

3 Evaluation Proposal

The evaluation should test whether intent-carrying leases reduce authority without destroying useful automation. The main workloads are: Skill workflows such as PDF extraction, spreadsheet generation, slide conversion, and code review; MCP workflows such as GitHub issue creation, calendar scheduling, Drive summarization, and ticket triage; mixed workflows that combine Skills, local scripts, MCP tools, and subagents; and adversarial workflows where PDFs, web pages, emails, issue comments, tool results, Skill instructions, or subagent summaries contain malicious instructions.

The baselines should separate the contribution from adjacent systems. A vanilla agent exposes all configured tools. A static allowlist grants workspace- or task-category permissions. A human-approval baseline relies on natural-language confirmation for high-impact actions. A skill-manifest baseline grants permissions at Skill granularity, following the SkillGuard family of designs. A tool-call guard checks direct invocations but not indirect subprocess behavior. An

OS-enforcement baseline applies sandbox or ActPlane-style rules after policy has been chosen, without INTENTCAP’s intent and context-authority model.

The main least-privilege metric is a risk-weighted authority score relative to expert-written oracle leases. Risk should include exposed tools, MCP methods, file-path breadth, network destinations, credential scopes, influence modes, delegation breadth, lease duration, and approval scope. The security metrics are attack success rate, wrong-sink actions, data exfiltration, privilege amplification, unauthorized approval inflation, malicious delegation, and high-impact decisions influenced by untrusted context. Utility metrics include benign task completion, output correctness, false denials, replans after denial, user approvals, and latency. Compiler metrics include invalid LLM lease proposals rejected by the checker, valid proposals rejected, evidence extraction precision and recall, proof completeness, and recovery after structured denials.

The most important ablation is the context-authority lattice. Without it, a system can still count tools and sandbox processes, but it cannot distinguish “PDF text may fill spreadsheet cells” from “PDF text may choose the GitHub repository.” Other ablations remove intent certificates, the control/data provenance split, lease minimization, the deterministic checker, runtime refinement, or the OS backend. The expected result is not that INTENTCAP prevents every prompt injection. It should show that, within modeled decision classes, the same untrusted context can still be useful as data while being structurally unable to authorize, delegate, choose sinks, or request broader power.

4 Positioning

INTENTCAP should be read as a capability compiler and checker for agent decisions. EIM and bpftime ask how an extension can safely use host-provided resources. ActPlane asks how policy can be enforced below the tool layer. SkillGuard asks how a Skill package can be governed as a permission-bearing artifact. INTENTCAP asks a different question: which future decisions may this context influence, under this user intent, and with what proof? That distinction is the paper’s center of gravity. It keeps the LLM outside the trusted computing base, treats the current user goal as the root of authority, and makes context influence a least-privilege capability rather than an implicit side effect of putting text in the prompt.

References

- [1] Model Context Protocol. 2025. Specification. <https://modelcontextprotocol.io/specification/2025-06-18>.
- [2] Model Context Protocol. 2025. Tools. <https://modelcontextprotocol.io/specification/2025-03-26/server/tools>.
- [3] OpenAI. 2026. Agent Skills. <https://developers.openai.com/codex/skills>. Accessed: 2026-07-02.
- [4] Shidong Pan, Xiaoyu Sun, Tianyi Zhang, Dianshu Liao, Meixue Si, and Zhenchang Xing. 2026. SkillGuard: A Permission Framework for Agent Skills. arXiv preprint arXiv:2606.03024. doi:10.48550/arXiv.2606.03024
- [5] Yusheng Zheng, Tianyuan Wu, Quanzhi Fu, Tong Yu, Wenan Mao, Wei Wang, Dan Williams, and Andi Quinn. 2026. ActPlane: Programmable OS-Level Policy Enforcement for Agent Harnesses. arXiv preprint arXiv:2606.25189. doi:10.48550/arXiv.2606.25189
- [6] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI ’25)*. USENIX Association. <https://www.usenix.org/conference/osdi25/presentation/zheng-yusheng>